

Missing Data Analysis in the Titanic Dataset

Introduction

This project begins with an exploration of missing data in the Titanic dataset. Understanding where and how much data is missing is a critical early step in any data science workflow. It helps inform decisions related to imputation, feature removal, or the need for alternative modeling strategies.

Step 1: Detecting Missing Values

Code	Output	
import pandas as pd	PassengerId	0
import matplotlib.pyplot as plt	Survived	0
	Pclass	0
<i># Load the dataset</i>	Name	0
df = pd.read_csv("Titanic-Dataset.csv")	Sex	0
	Age	177
<i># Show the count of missing data</i>	SibSp	0
missing_data = df.isnull().sum()	Parch	0
	Ticket	0
<i># Calculate the percentage of missing data</i>	Fare	0
missing_percentage = (missing_data / len(df)) * 100	Cabin	687
	Embarked	2
print (missing_data)	dtype: int64	
print (missing_percentage)		
	PassengerId	0.000000
	Survived	0.000000
	Pclass	0.000000
	Name	0.000000
	Sex	0.000000
	Age	19.865320
	SibSp	0.000000
	Parch	0.000000
	Ticket	0.000000
	Fare	0.000000
	Cabin	77.104377
	Embarked	0.224467
	dtype: float64	

Interpretation

The code above uses the `isnull()` function to count missing values for each column in the dataset and then computes their corresponding percentages. This step provides essential insight into the quality of each feature before any cleaning or modeling.

Key findings from the output:

- **Cabin:** 77.1% missing values — likely too incomplete to be useful without significant imputation or restructuring.
- **Age:** 19.9% missing values — a critical feature, but manageable via techniques like mean, median, or predictive imputation.
- **Embarked:** only 0.22% missing — this can be easily handled with simple imputation or row removal.

The remaining columns are complete, providing a solid foundation for continued analysis.

Step 2: Methods for Imputing Missing Data

Code

```
# Methods for imputing missing data

# Method 1: Drop rows with any missing values
df_dropna = df.dropna()

# Method 2: Fill missing 'Age' values with median
df_median_fill = df.copy()
df_median_fill['Age'].fillna(
    df_median_fill['Age'].median(), inplace=True)

# Method 3: Fill missing 'Embarked' values with mode
df_mode_fill = df.copy()
df_mode_fill['Embarked'].fillna(
    df_mode_fill['Embarked'].mode()[0], inplace=True)
```

Explanation:

This section demonstrates three widely used methods for handling missing data in a pandas DataFrame. Each approach handles null values differently, allowing flexibility based on the context of the analysis:

- **Method 1: `dropna()`**
This removes all rows that contain at least one missing value, producing the `df_dropna` DataFrame. While this ensures a clean dataset, it may significantly reduce the data volume if missing values are common. This method is best when missingness is sparse or when working with a large dataset where losing some rows is acceptable.
- **Method 2: Median Imputation for Age**
Here, a copy of the dataset is created, and missing values in the `Age` column are filled using the median. This is effective for numerical data, especially when the distribution is skewed or contains outliers. Using the median preserves the central tendency without being overly influenced by extreme values.
- **Method 3: Mode Imputation for Embarked**
Another copy of the dataset is made, and missing values in the categorical `Embarked` column are

replaced with the most frequent value (mode). This technique assumes that the most common category represents the best guess for missing entries.

Each method modifies a copy of the original dataset, leaving the source data intact. This approach is ideal when you want to compare multiple preprocessing strategies or avoid irreversible changes to your raw data.

Overall, these imputation techniques are essential steps in preparing data for further analysis or machine learning models. The choice of method depends on the data type, amount of missingness, and modeling goals. Thoughtful handling of missing data helps maintain data integrity and improves the reliability of downstream results.

Step 3: Dropping Unnecessary or Mostly Empty Columns

Code

```
# Drop unnecessary or mostly empty columns
df_cleaned = df.drop(columns=[
    'PassengerId', 'Cabin', 'Name', 'Ticket'
])
```

Explanation:

This step involves dropping columns that are either not useful for predictive analysis or have a high proportion of missing data. The purpose is to simplify the dataset and retain only the most informative features for modeling.

- **Cabin:** Contains over 77% missing values. With such a large portion of data missing, it is not practical to include this feature without significant imputation or engineering.
- **Name, Ticket, PassengerId:** These are identifier columns. While they are unique to each passenger, they do not represent features that could logically affect the likelihood of survival. Including them might introduce noise or lead to overfitting in some models.

By dropping these columns, the resulting `df_cleaned` DataFrame becomes leaner, more interpretable, and better suited for downstream machine learning tasks. This is a common and important step in the data preprocessing pipeline to ensure that only relevant and high-quality features are used in analysis or model training.

Feature Types and Preprocessing

In the Titanic dataset, features can be categorized into two main types: numerical and categorical (string/object).

- **Numerical features:** These include columns like `Age`, `Fare`, `SibSp`, and `Parch`, which contain either continuous or discrete values and support mathematical operations such as calculating the mean or standard deviation.
- **Categorical features:** These include columns such as `Sex`, `Embarked`, and `Pclass`, which represent different categories or labels. While `Pclass` is stored as an integer, it is still considered a categorical feature. Categorical features are generally stored as string or object types in pandas.

Understanding this distinction is crucial for preprocessing. Categorical data must be converted into numerical form before being used in machine learning models. This conversion is typically done using:

- **Label encoding:** This method assigns a unique integer to each category, making it suitable for binary or ordinal categories.
- **One-hot encoding:** This technique creates a separate binary column for each category and is preferred when the categories do not have an ordinal relationship.

To prepare the data for modeling, several preprocessing steps are required:

- Encoding categorical features using one-hot or label encoding
- Handling missing values, such as imputing missing values in the **Age** or **Embarked** columns
- Feature scaling (normalization or standardization) for numerical features like **Age** and **Fare**, especially when using models such as logistic regression

Normalization scales features to a range between $[0, 1]$, while standardization transforms features to have a mean of 0 and a standard deviation of 1:

$$x_{\text{standardized}} = \frac{x - \mu}{\sigma} \quad \text{and} \quad x_{\text{normalized}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

These scaling techniques help models converge more quickly and improve their performance.

Step 4: Complete Feature Encoding and Scaling Pipeline

Code

```
from sklearn.preprocessing import LabelEncoder, StandardScaler

# Load the dataset
df_final = pd.read_csv('Titanic-Dataset.csv')

# Drop columns that are not useful for modeling
df_final = df_final.drop(columns=['PassengerId', 'Name', 'Ticket', 'Cabin'])

# Handle missing values
df_final['Age'] = df_final['Age'].fillna(df_final['Age'].median())
df_final['Embarked'] = df_final['Embarked'].fillna(df_final['Embarked'].mode()[0])

# Encode 'Sex' using LabelEncoder
le = LabelEncoder()
df_final['Sex'] = le.fit_transform(df_final['Sex'])

# One-hot encode 'Embarked'
df_final = pd.get_dummies(df_final, columns=['Embarked'], drop_first=True)

# Standardize numerical features
scaler = StandardScaler()
df_final[['Age', 'Fare']] = scaler.fit_transform(df_final[['Age', 'Fare']])

print(df_final.head())
```

Output: First 5 Rows of Preprocessed Dataset

	Survived	Pclass	Sex	Age	SibSp	Parch	Fare	Embarked_Q	Embarked_S
0	0	3	1	-0.565736	1	0	-0.502445	False	True
1	1	1	0	0.663861	1	0	0.786845	False	False
2	1	3	0	-0.258337	0	0	-0.488854	False	True
3	1	1	0	0.433312	1	0	0.420730	False	True
4	0	3	1	0.433312	0	0	-0.486337	False	True

The above output confirms that the preprocessing was successful. All categorical columns have been encoded into numerical or boolean values, and numerical features such as **Age** and **Fare** have been standardized. The dataset now consists only of clean, numerical features—making it suitable for training a variety of machine learning models.

Explanation:

This code block performs a comprehensive data preprocessing pipeline to prepare the Titanic dataset for machine learning models. It includes all critical steps such as cleaning, encoding, and scaling, ensuring the final dataset is numerical, clean, and normalized.

- **Dropping Unnecessary Columns:** The columns `PassengerId`, `Name`, `Ticket`, and `Cabin` are removed since they either provide no predictive value or contain excessive missing data. This reduces dimensionality and improves modeling efficiency.
- **Handling Missing Values:**
 - `Age` is filled with the median value, which is robust to outliers and preserves the data distribution.
 - `Embarked` is filled with the mode, the most common value in the column, making it a logical replacement for a small number of missing entries.
- **Encoding Categorical Features:**
 - `Sex` is label-encoded, converting it into binary format (e.g., male = 1, female = 0), which is suitable for models that work with numerical input.
 - `Embarked` is one-hot encoded using `pd.get_dummies()`, and `drop_first=True` is used to avoid multicollinearity. This results in two new binary columns (e.g., `Embarked_Q`, `Embarked_S`).
- **Feature Scaling:** `StandardScaler` is applied to the `Age` and `Fare` columns, which transforms them to have a mean of 0 and a standard deviation of 1. This is particularly important for algorithms sensitive to feature scale, such as logistic regression or support vector machines.

The result is a fully cleaned and preprocessed dataset, stored in `df_final`, that is ready for use in training machine learning models. This preprocessing ensures that all features are numeric, appropriately scaled, and free from missing data—essential prerequisites for accurate and stable model performance.